LEARNING NOTE / 01

# 내 로봇 AI 개발 환경 이해하고 공부하기

# 설치 목록에서 첫 GPU 실습까지

### I Want RoboRobo / 개인 사이드 프로젝트

기준일 2026-10-04 · Windows 데스크톱 · Python 3.12.15 · RTX 3060 12GB

이 문서는 PyTorch를 다시 공부하면서 로봇 시뮬레이션과 모방학습을 시작하는 사람을 위한 안내서입니다. 주 5-6시간, 6-8주를 기준으로 구성했습니다. 패키지 이름을 외우기보다 '어디에 설치되고, 어떤 데이터를 주고받으며, 무엇을 확인하면 작동했다고 말할 수 있는가'를 익히는 것이 목표입니다.

### 현재 확인된 것

GPU 계산과 역전파, MuJoCo 물리 계산과 RGB 렌더링, Panda 집기 환경 초기화와 1스텝, 영상 생성과 읽기가 통과했습니다. 프로젝트 가상환경에는 의존성을 포함해 178개 Python 배포 패키지가 설치되어 있습니다.

### 앞으로 구현할 것

시연 수집, ACT 모델 학습, 실제 집기 성공률 평가, 추론 서비스, ROS 2 연동은 앞으로 할 작업입니다. 지금 환경 검사 결과는 학습된 로봇의 성능을 의미하지 않습니다.

| 읽을 부분 | 이해할 내용 |
|---|---|
| 2-5쪽 | 개발 환경의 구조, VS Code, Python, Git과 아이패드 |
| 6-10쪽 | 텐서와 학습, 로봇 환경, ACT, 데이터, 추론 서비스 |
| 11-13쪽 | 따라 하는 실습, 8주 공부 계획, 문제 해결 |
| 14-18쪽 | 용어 사전, 설치 버전, 전체 패키지 목록, 공식 자료 |

권장 사용법: 먼저 2-5쪽을 읽고 11쪽 GPU 실습을 해보세요. 로봇 용어가 낯설면 7쪽과 14쪽을 다시 확인하세요. 전체 패키지 목록은 필요할 때 찾아보는 부록입니다.

LEARNING NOTE / 02

# 환경은 여러 층으로 이루어진다

VS Code를 설치했다고 Python이나 GPU 학습 기능이 자동으로 설치되지는 않습니다. 아래 각 층이 연결되어야 코드가 실행됩니다.

**1. 운영체제와 하드웨어**
Windows + RTX 3060 + NVIDIA 드라이버

**2. 개발 프로그램**
VS Code / Git / uv / FFmpeg

**3. 편집기 확장**
한국어 / PyCharm 키맵 / Python / Pylance / Jupyter / Ruff

**4. 프로젝트 Python 환경**
Python 3.12.15 + 프로젝트 전용 .venv

**5. Python 라이브러리**
PyTorch / LeRobot / MuJoCo / Gymnasium / FastAPI 등

**6. 내 프로젝트**
시연 수집 -> 학습 -> 평가 -> 추론 서비스 -> 작업 관리

## 가장 많이 혼동하는 세 가지

| 구분 | 예시 | 설치·설정 위치 |
|---|---|---|
| 프로그램 | VS Code, Git, FFmpeg | Windows에서 실행하는 앱 또는 명령 |
| VS Code 확장 | Pylance, 한국어, 키맵 | VS Code 기능을 추가하는 확장 폴더 |
| Python 패키지 | torch, numpy, mujoco | 이 프로젝트의 .venv 안 |

Jupyter도 두 부분이 있습니다. VS Code의 Jupyter 확장은 노트북 화면을 제공하고, .venv의 ipykernel은 셀의 Python 코드를 실제로 실행합니다. 두 환경 선택이 맞아야 합니다.

### 스스로 설명해 보기

'Pylance를 설치했으니 torch를 import할 수 있다'는 말이 왜 틀릴까요? Pylance는 코드 분석 도구이고, torch는 실행할 Python 환경에 따로 설치하는 라이브러리이기 때문입니다.

# VS Code에서 무엇을 설치했나

| 확장 / 상태 | 역할 |
| --- | --- |
| 한국어 언어 팩 / 새로 설치 | 메뉴와 안내 문구를 한국어로 표시. locale=ko 저장 완료. |
| IntelliJ IDEA Keybindings / 새로 설치 | PyCharm과 같은 JetBrains 계열의 주요 단축키 적용. |
| Ruff / 새로 설치 | Python 오류 후보, import 정리, 코드 서식 검사. |
| GitHub Pull Requests / 새로 설치 | VS Code 안에서 GitHub PR과 이슈 작업 지원. |
| Python / 기존 설치 확인 | Python 실행, 인터프리터 선택, 테스트 기능 연결. |
| Pylance / 기존 설치 확인 | 자동완성, 타입 분석, 정의 이동. |
| Python Debugger / 기존 설치 확인 | 중단점을 걸고 변수와 실행 흐름 확인. |
| Python Environments / 기존 설치 확인 | 가상환경 검색과 선택, 패키지 관리. |
| Jupyter / 기존 설치 확인 | .ipynb 파일 열기와 셀 실행. 관련 보조 확장도 있음. |
| WSL / 기존 설치 확인 | Windows에서 Linux 환경에 연결. WSL 운영체제 설치와는 별개. |

## 먼저 익힐 단축키

| 키 | 기능 |
| --- | --- |
| Ctrl + Shift + P | 명령 팔레트 열기. Python: Select Interpreter 등을 검색. |
| Ctrl + D / Ctrl + / | 현재 줄 복제 / 주석 전환 |
| Ctrl + Alt + L / Shift + F6 | 코드 서식 정리 / 이름 변경 |
| Ctrl + B / Shift + F9 | 정의 이동 / 디버깅 시작 |

키맵은 주요 단축키를 맞추며 PyCharm의 모든 전용 기능을 구현하지는 않습니다. 확장 또는 OS 단축키 충돌은 키보드 바로 가기 화면에서 확인하세요.

### 첫 실행 설정

열려 있는 VS Code를 완전히 닫고 다시 열면 한국어가 적용됩니다. 작업 폴더는 C:/Users/user/Projects/I_Want_RoboRobo 를 사용하세요. Python 인터프리터는 .venv/Scripts/python.exe, 노트북 커널은 Python (Robot Learning Lab)을 선택하면 됩니다.

# Python · 가상환경 · uv 이해하기

**인터프리터**는 .py 코드를 실행하는 Python 프로그램입니다. **가상환경**은 프로젝트마다 사용하는 Python과 라이브러리 조합을 분리하는 폴더입니다. 기존 Miniconda 환경을 유지하면서 이번 프로젝트용 .venv를 따로 만들었습니다.

| 파일 / 도구 | 뜻과 역할 |
|---|---|
| .venv/ | 설치된 라이브러리와 실행 파일. 크고 PC에 종속되므로 Git에 올리지 않음. |
| .python-version | 이 프로젝트에서 사용할 Python 버전: 3.12.15 |
| pyproject.toml | 프로젝트가 직접 요구하는 패키지와 허용 버전, Ruff 설정. |
| uv.lock | 간접 의존성까지 해결한 버전 기록. 함께 커밋해서 환경 재현에 사용. |
| uv | Python 버전과 가상환경, 패키지 의존성을 관리하는 도구. |
| pip / conda | 다른 설치·환경 관리 도구. 이번 프로젝트의 기준은 uv와 .venv. |

## 의존성은 무엇인가

LeRobot을 직접 설치하면 LeRobot이 필요로 하는 datasets, safetensors 같은 패키지도 함께 설치됩니다. 내가 직접 지정한 것은 **직접 의존성**, 그것들이 요구하는 것은 **간접 의존성**입니다. 178개를 모두 외울 필요는 없습니다.

## 현재 PC에서 쓰는 명령 - PowerShell

```
cd "$env:USERPROFILE/Projects/I_Want_RoboRobo"
.\.venv\Scripts\python.exe --version
uv sync --locked --python .venv/Scripts/python.exe --no-python-downloads
.\.venv\Scripts\python.exe scripts/check_environment.py
```

uv sync는 선언과 잠금 파일에 맞춰 환경을 동기화합니다. --locked는 잠금 파일 변경이 필요한 상황에서 멈추게 합니다. 위 --python 경로는 이 PC에 이미 만들어 둔 Python을 명시합니다. 새 PC에서는 먼저 Python 3.12.15와 .venv를 준비해야 합니다.

### 실수 줄이기

터미널에서 그냥 python을 실행하면 기존 Miniconda Python이 선택될 수 있습니다. sys.executable로 실제 경로를 확인하거나 위처럼 .venv의 python.exe를 명시하세요. 이 가상환경에서는 uv로 패키지를 관리하세요.

LEARNING NOTE / 05

# GitHub · 로컬 PC · 아이패드

| 개념 | 이번 프로젝트에서는 |
|---|---|
| Git | 내 PC에서 변경 이력을 관리하는 프로그램. |
| GitHub | Git 저장소를 온라인에서 보관하고 보여주는 서비스. |
| 조직 / 저장소 | 조직 I-Wani-RoboRobo / 저장소 I_Want_RoboRobo |
| commit | 변경한 코드를 설명과 함께 로컬 이력에 저장. |
| push / pull | 로컬 커밋을 원격에 보내기 / 원격 변경을 가져오기. |
| branch / PR | 별도 작업 흐름 / 변경을 검토하고 합치기 위한 요청. |
| .gitignore | 가상환경, 영상, 체크포인트, 비밀값 등 제외할 파일을 지정. |

## 카카오톡에 붙여 둘 아이패드용 주소

https://vscode.dev/github/I-Wani-RoboRobo/I_Want_RoboRobo

Safari에서 위 주소를 열면 저장소를 VS Code 웹 편집기로 볼 수 있습니다. GitHub에 로그인하면 권한 범위 안에서 편집과 커밋을 할 수 있습니다. 아직 코드 파일이 없으면 폴더나 파일이 적게 보일 수 있습니다.

| 아이패드 웹에서 | 데스크톱에서 |
|---|---|
| 코드 읽기, 문서 공부, 가벼운 수정 | Python 실행, GPU 학습, 시뮬레이션 |
| 웹 환경의 언어·확장 별도 설정 | 이번에 설치한 한국어·키맵·.venv 사용 |

vscode.dev는 데스크톱의 .venv와 RTX 3060을 자동으로 사용하지 않습니다. 브라우저만 열어서 이 Python 로봇 환경을 실행할 수는 없습니다. 원격 실행 환경 연결은 추후 별도 작업입니다.

## 작업 뒤 남기는 최소 기록

```
git status
git diff
git add scripts/example.py
git commit -m "Add tensor practice"
git push
```

위 example.py는 앞으로 만들 예시 파일명입니다. git diff로 내용을 보고 필요한 파일만 추가하세요. 아이패드에서 수정했다면 데스크톱 수정 전에 git pull로 변경을 가져오세요.

# PyTorch와 GPU 학습의 핵심

PyTorch는 숫자 배열을 계산하고 신경망을 학습하는 라이브러리입니다. **텐서(tensor)**는 여러 차원의 숫자 배열입니다. 로봇 이미지, 관절 상태, 행동 모두 텐서로 바뀌어 모델에 들어갑니다.

| 개념 | 예시와 이해할 점 |
|---|---|
| shape | [8, 4] = 샘플 8개, 각 샘플의 값 4개. 축마다 무엇을 의미하는지 설명하기. |
| dtype | float32, int64 등 숫자 표현 방식. 입력과 모델이 요구하는 타입 맞추기. |
| device | cpu 또는 cuda. 입력과 모델을 같은 장치에 두기. |
| batch | 한 번에 학습하는 샘플 묶음. 크면 VRAM 사용량도 대체로 증가. |
| loss | 예측이 정답에서 얼마나 벗어났는지 나타내는 값. |
| gradient / backward | 파라미터를 바꾸면 loss가 어떻게 변하는지 계산 / 역전파. |
| optimizer / learning rate | gradient로 파라미터를 갱신하는 방법 / 갱신 폭을 정하는 값. |

## 한 번의 학습 업데이트

```
optimizer.zero_grad()
prediction = model(inputs)
loss = loss_fn(prediction, targets)
loss.backward()
optimizer.step()
```

zero_grad는 이전 gradient를 지우고, model은 예측을 만들고, backward는 gradient를 계산하고, step은 파라미터를 갱신합니다. 이 반복과 데이터 공급이 학습의 기본 구조입니다.

### GPU 환경에서 확인한 값

RTX 3060, VRAM 12GB. 설치된 PyTorch는 2.11.0+cu128이며 포함된 CUDA 런타임은 12.8입니다. 실제 작은 Linear 모델에서 GPU 예측, 역전파, Adam 업데이트가 통과했습니다. ACT 전체 학습의 메모리 사용량과 성능은 아직 측정하지 않았습니다.

NVIDIA 드라이버, PyTorch에 포함된 CUDA 런타임, 별도 CUDA Toolkit은 다른 구성 요소입니다. 현재 패키지의 일반 학습·추론에는 별도 Toolkit이 필요하지 않았습니다. CUDA 코드를 직접 컴파일하는 작업이 생기면 그때 검토하세요.

# 로봇 시뮬레이션을 읽는 방법

**MuJoCo**는 로봇의 운동과 접촉을 계산하는 물리 엔진입니다. **Gymnasium**은 환경의 시작과 한 단계 진행을 다루는 공통 인터페이스입니다. **gym-hil**은 이번에 사용한 Panda 집기 환경 등을 제공합니다.

| 환경 용어 | 의미 |
| --- | --- |
| observation / 관측 | 현재 얻을 수 있는 정보. 상태 값, 카메라 이미지 등이 될 수 있음. |
| action / 행동 | 환경에 전달하는 제어 입력. 단위·차원·범위를 먼저 확인해야 함. |
| reset / 초기화 | 새 에피소드 시작. seed로 초기 조건의 재현을 돕기. |
| step / 한 단계 | 행동을 적용하고 다음 관측, reward, 종료 정보 받기. |
| episode / 에피소드 | 초기화부터 종료까지의 한 번의 시도. |
| reward / 보상 | 환경이 계산한 값. 별도로 정의한 성공률과 항상 같지는 않음. |
| terminated / truncated | 환경의 종료 조건 / 시간 제한 등 외부 이유에 의한 종료. |

## 로봇의 한 단계

```
observation, info = env.reset(seed=0)
action = policy(observation)
next_obs, reward, terminated, truncated, info = env.step(action)
```

policy는 관측을 보고 행동을 정하는 함수 또는 모델입니다. 처음에는 사람이 직접 행동을 정해도 됩니다. sim step의 시간, action의 의미, 이미지 갱신 간격이 데이터 수집과 학습에서 일치해야 합니다.

### 현재 환경 검사에서 실제로 나온 것

gym_hil/PandaPickCubeBase-v0를 초기화하고 0 행동으로 한 단계 진행했습니다. 관측 키는 agent_pos와 environment_state였습니다. 별도 MuJoCo 검사에서는 64 x 64 x 3 RGB 이미지를 생성했습니다. 선택한 집기 환경에 학습용 카메라 관측을 붙이는 작업은 앞으로 확인해야 합니다.

## 먼저 알아야 할 로봇 단어

관절(joint)은 회전·이동하는 연결부입니다. 엔드 이펙터(end effector)는 로봇 끝의 도구이며 여기서는 그리퍼입니다. pose는 위치와 자세를 함께 뜻합니다. 좌표계(frame)는 위치·방향을 표현하는 기준이며 카메라, 로봇 바닥, 도구 좌표가 서로 다를 수 있습니다.

# LeRobot · 모방학습 · ACT

**모방학습(imitation learning)**은 사람 또는 기존 제어기가 보여준 관측-행동 예시로 정책을 배우는 방법입니다. **행동 복제(behavior cloning)**는 관측에서 시연 행동을 예측하도록 학습합니다. 이번 프로젝트는 이 흐름에서 시작합니다.

**LeRobot**은 로봇 학습용 데이터, 정책과 학습 도구를 제공하는 라이브러리입니다. 설치한 0.6.1에서 ACT 모델 클래스와 데이터셋 모듈 import를 확인했습니다. 데이터 준비와 학습 실행은 별도 단계입니다.

## ACT가 하는 일

ACT는 Action Chunking with Transformers의 약자입니다. 한 번에 행동 하나만 예측하는 대신 여러 시점의 행동 묶음을 예측합니다. 이미지를 비롯한 관측과 로봇 상태를 받아 행동 시퀀스를 만드는 것이 핵심입니다. 행동 묶음 중 무엇을 얼마나 실행할지는 제어 루프 설정에 따라 달라집니다.

**1. 시연 수집**
사람이 집기 -> 관측, 행동, 시각, 종료 상태 저장

**2. 데이터 정리**
에피소드 단위 분리 -> 관측·행동 대응 확인 -> 정규화

**3. ACT 학습**
입력 관측 -> 예측 행동 묶음 -> loss 계산 -> 업데이트

**4. 새 초기 조건에서 평가**
실제 환경에 행동 적용 -> 성공 여부, 지연, 실패 기록

| 추가 용어 | 뜻 |
|---|---|
| chunk size | 모델이 한 번에 예측하는 행동 시퀀스 길이. |
| normalization | 값 범위를 학습하기 편하게 변환. 학습과 추론에서 같은 기준 사용. |
| checkpoint | 학습 중 저장한 가중치와 상태. 입력·정규화 설정도 함께 기록. |
| overfitting | 학습한 시연에는 잘 맞지만 새로운 상황에서는 성능이 떨어지는 현상. |

### 평가에서 꼭 구분할 것

학습 loss가 낮다는 사실만으로 집기가 성공했다고 판단할 수 없습니다. 같은 평가 seed와 시간 제한을 정하고, 새 초기 조건에서 성공 횟수 / 전체 시도 수를 기록해야 합니다.

# 데이터와 영상 패키지의 역할

| 도구 | 어디에 쓰나 |
| --- | --- |
| NumPy | 관측·행동 배열의 shape, 범위, 통계 확인. |
| pandas | 에피소드 목록과 결과표 같은 표 데이터 분석. |
| SciPy | 수치·통계 계산. 필요한 기능부터 찾아 쓰기. |
| scikit-learn | 기초 ML, 지표·전처리 도구. 첫 ACT 구현의 중심은 PyTorch. |
| Matplotlib | loss 곡선, 성공률 비교, 행동 값 그래프. |
| datasets / PyArrow | 데이터셋과 컬럼형 데이터 저장·읽기를 지원하는 의존성. |
| ImageIO / OpenCV / av | 이미지와 영상 처리. OpenCV는 headless 버전이므로 자체 GUI 기능 없음. |
| FFmpeg / TorchCodec | 영상 인코딩·디코딩 프로그램 / PyTorch에서 영상 읽기. |
| safetensors / Hugging Face Hub | 가중치 파일 저장·읽기 / 데이터·모델 배포 관련 도구. |
| W&B / Accelerate | 실험 기록 / 학습 실행 지원. 설치되어 있지만 계정 연결·사용은 별도. |

## 시연 하나에 기록할 최소 항목

에피소드 ID, 시점(timestamp 또는 frame index), 관측, 로봇 상태, 행동, 종료·성공 정보, FPS와 제어 주기를 정합니다. 이미지와 행동이 서로 다른 시점이면 잘못된 대응을 학습할 수 있습니다.

## 데이터 누수를 막는 방법

한 에피소드의 연속 프레임을 학습과 검증에 섞으면 비슷한 장면이 양쪽에 들어갑니다. 에피소드 단위로 먼저 분리하세요. 정규화 통계도 학습 데이터에서 구하고 검증·평가에 그대로 적용합니다.

### 영상 환경을 왜 맞췄나

현재 FFmpeg shared 8.1.2와 TorchCodec 0.11.1 조합으로 1초짜리 영상을 생성하고 첫 프레임을 읽는 검사를 통과했습니다. shared 배포의 DLL이 영상 로더에 필요합니다. 이 결과는 실제 LeRobot 시연 데이터셋 전체 저장·재생 검증과는 구분합니다.

# 추론 서비스와 ROS 2로 이어가기

**학습(training)**은 데이터로 모델 가중치를 바꾸는 과정입니다. **추론(inference)**은 학습된 모델로 입력에서 행동을 예측하는 과정입니다. **서빙(serving)**은 이 추론을 다른 프로그램이 호출할 수 있는 형태로 운영하는 일입니다.

| 구성 | 역할 / 현재 상태 |
| --- | --- |
| FastAPI | 입력과 출력을 정의하는 HTTP API. 패키지 설치, import 확인 완료. |
| Uvicorn | FastAPI 애플리케이션을 실행하는 서버. 설치 완료. |
| Pydantic | API 입력·출력의 구조와 값 검사. FastAPI의 의존성으로 설치됨. |
| ROS 2 | 로봇 프로그램 간 통신과 작업 관리. 이번 설치에 포함하지 않았음. |
| WSL | Windows 안의 Linux 실행 환경. 확장 설치 확인만 했으며 환경 구축은 추후 작업. |

## 서비스에서 합의할 입력과 출력

입력: 모델 버전, 관측 시각, 이미지, 로봇 상태, 요청 ID. 출력: 행동 시퀀스, 사용한 모델 버전, 처리 시간. shape, dtype, 단위, 좌표계와 정규화 규칙을 함께 정해야 합니다. HTTP에서 큰 이미지나 배열을 전달하는 방식도 별도로 설계해야 합니다.

## 속도와 실패도 제품 동작이다

latency는 요청부터 응답까지 걸린 시간, throughput은 단위 시간에 처리한 요청 수입니다. 로봇은 오래된 관측의 행동을 뒤늦게 실행하면 문제가 생길 수 있으므로 timeout과 취소, 실패 시 행동을 어떻게 멈출지 정의해야 합니다.

| ROS 2 개념 | 앞으로의 활용 |
| --- | --- |
| node | 시뮬레이터, 모델 호출, 작업 관리 등을 담당하는 실행 단위. |
| topic | 관측·상태처럼 계속 전달되는 메시지 흐름. |
| service | 한 번 요청하고 응답을 받는 통신. |
| action | 집기처럼 시간이 걸리는 작업. 진행 정보와 취소를 다룸. |

## 최종 프로젝트와 연결해서 설명하기

고객 피드백 자동화와 로봇 학습은 도메인이 다르지만 입력 계약, 모델 버전, 실패 처리, 평가, 지연 측정이라는 엔지니어링 문제를 공유합니다. 로봇 팔 한 개와 policy 한 개로 시작하고, 작업 관리를 확장하면서 이런 역량을 보여주세요.

# 오늘 해볼 실습 - 60분

## 1. 실행할 환경 확인 - 10분

```
cd "$env:USERPROFILE/Projects/I_Want_RoboRobo"
.\.venv\Scripts\python.exe -c "import sys; print(sys.executable)"
.\.venv\Scripts\python.exe scripts/check_environment.py
```

기대 결과: 실행 경로가 프로젝트 .venv를 가리키고 검사 항목이 모두 PASS. 결과는 outputs/environment-check.json에 저장됩니다. uv나 FFmpeg가 안 보이면 새 VS Code와 새 터미널로 다시 열어 PATH 갱신을 확인하세요.

## 2. GPU에서 작은 모델 학습 - 25분

experiments/tensor_practice.py 파일을 새로 만들고 아래 코드를 넣으세요. 새 파일 예제이며 아직 저장소에 이 실습 구현은 추가하지 않았습니다.

```
import torch

torch.manual_seed(0)
device = "cuda" if torch.cuda.is_available() else "cpu"
x = torch.randn(64, 4, device=device)
y = x.sum(dim=1, keepdim=True)
model = torch.nn.Linear(4, 1).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.03)

for step in range(100):
    optimizer.zero_grad()
    prediction = model(x)
    loss = (prediction - y).square().mean()
    loss.backward()
    optimizer.step()
    if step % 20 == 0:
        print(step, device, float(loss.detach().cpu()))

.\.venv\Scripts\python.exe experiments/tensor_practice.py
```

설명할 것: x의 shape은 [64, 4], y와 출력은 [64, 1]입니다. loss가 처음보다 줄어드는지 확인하세요. 같은 샘플에 반복해서 맞추는 연습이므로 일반화 성능을 보여주는 실험은 아닙니다.

## 3. 바꾸고 설명하기 - 15분

batch를 64에서 16으로 바꾸고 shape을 출력해 보세요. learning rate를 0.03에서 0.003으로 바꿔 loss 감소 속도를 비교하세요. 마지막으로 CPU로 바꿔 실행하고 데이터와 모델이 함께 이동해야 하는 이유를 설명하세요.

## 4. 공부 기록 남기기 - 10분

오늘의 Python 경로, tensor shape, 처음과 마지막 loss, 바꾼 값, 궁금한 점을 적으세요. 공개 저장소에 올릴 때는 코드와 짧은 설명만 남기면 충분합니다.

# 주 5-6시간으로 공부하는 순서

매주 개념 읽기 1시간, 직접 구현 3시간, 결과 확인 1시간, 기록 30분 정도로 나누세요. 개념을 전부 이해한 뒤 코딩하려고 기다리기보다 작은 실행 결과를 먼저 만드세요.

| 주차 | 공부와 구현 | 완료를 보여주는 결과 |
|---|---|---|
| 1주 | .venv, Git, tensor, reset/step. 시뮬레이터 조작 확인. | GPU 실습 설명 + 환경 검사 + 시연 1개 저장 |
| 2주 | observation/action, 에피소드, FPS, 데이터 shape. | 시연 데이터 목록 + 이미지·행동 대응 확인 |
| 3주 | 학습 루프, ACT 입력, loss, checkpoint. | 작은 배치로 첫 학습 + 설정·로그·체크포인트 |
| 4주 | 검증 분리, seed, 성공 판정, 실패 유형. | 새 초기 조건에서 첫 성공률 측정 |
| 5주 | 데이터 수 또는 설정 2개 비교. | 같은 평가 조건의 결과표 + 실패 영상 |
| 6주 | FastAPI, 입력 계약, latency, timeout. | 실제 policy가 연결된 추론 호출과 지연 기록 |
| 7주 | 필요하면 Linux/ROS 2와 작업 action. | 시작·진행·결과·취소가 동작하는 연동 |
| 8주 | 통합 시나리오, 재현 문서, 데모. | 다른 사람이 따라 할 실행법과 한계 설명 |

## 지금 먼저 / 구현하면서 / 뒤로 미뤄도 되는 것

**먼저:** 인터프리터와 가상환경, Git, shape/dtype/device, 학습 루프, observation/action, 에피소드 분리.
**구현하면서:** ACT 구조, 정규화, 카메라 데이터, 성공률과 지연 측정.
**뒤로:** 고급 CUDA 최적화, 분산 학습, 여러 로봇과 여러 policy, 복잡한 클라우드 배포.

### 취업 때 보여줄 엔지니어링 근거

설치한 패키지 개수보다 재현 가능한 실행법, 새 조건에서의 평가, 실패 원인과 개선, 모델과 데이터 버전, 실제 서비스 지연을 보여주는 편이 좋습니다. README에서 구현 완료와 계획을 구분하고 수치는 실제 측정한 것만 적으세요.

# 문제가 생겼을 때 확인 순서

| 증상 | 먼저 확인할 것 |
|---|---|
| ModuleNotFoundError | sys.executable 확인. 패키지가 설치된 .venv로 실행하는지 보기. |
| VS Code 노트북만 import 실패 | 우측 상단 커널이 Python (Robot Learning Lab)인지 확인. |
| torch.cuda.is_available()가 False | nvidia-smi와 Python 경로 확인. 설치 torch 버전의 +cu128 확인. |
| CUDA out of memory | batch나 이미지 크기를 줄여 확인. 실제 메모리 사용을 기록. |
| 텐서 device/shape 오류 | 입력·정답·모델의 device와 shape을 출력. 기대 차원과 비교. |
| MuJoCo 파일을 못 찾음 | 이 PC에서는 영문 연결 경로의 .venv로 실행. 원래 파일이 존재하는지도 확인. |
| TorchCodec DLL / FFmpeg 오류 | 새 터미널에서 ffmpeg -version 확인. 검사 스크립트의 DLL 경로 설정 참고. |
| uv / git 명령을 찾을 수 없음 | 새 터미널로 PATH 갱신. 앱을 설치한 경로와 Get-Command 결과 비교. |
| 한국어가 적용되지 않음 | VS Code 완전 종료 후 재실행. 표시 언어 Korean / ko 확인. |
| 아이패드에서 Python 실행 안 됨 | vscode.dev의 편집 환경과 데스크톱 실행 환경을 구분. |

## 한글 경로를 해결한 방식

C:/Users/user/Projects/I_Want_RoboRobo 는 원래 프로젝트 폴더를 가리키는 Windows junction입니다. 복사본이 아니므로 어느 경로에서 수정해도 같은 파일입니다. 이 PC에서 MuJoCo가 한글이 포함된 XML 경로를 읽지 못해 영문 경로를 준비했습니다. 프로젝트와 노트북 커널은 영문 경로를 사용하세요.

## 도움을 요청할 때 붙이면 좋은 정보

```
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -c "import sys; print(sys.executable)"
.\.venv\Scripts\python.exe -c "import torch; print(torch.__version__)"
nvidia-smi
ffmpeg -version
git status
```

실행한 명령, 오류의 마지막 부분만이 아니라 원인까지 포함한 traceback, 방금 바꾼 설정을 함께 남기면 원인을 찾기 쉽습니다. 비밀값이나 토큰이 포함되어 있으면 공유 전에 제거하세요.

# 찾아보는 용어 사전

| 용어 | 한 문장 뜻 |
|---|---|
| runtime / interpreter | 프로그램을 실행하는 환경 / Python 코드를 실행하는 프로그램. |
| extension / package | 편집기에 기능을 더하는 확장 / Python에서 쓰는 라이브러리 배포 단위. |
| dependency / lockfile | 필요한 다른 패키지 / 해결된 의존성 버전 기록. |
| PATH / DLL | 명령 실행 파일을 찾는 경로 목록 / Windows 공유 라이브러리 파일. |
| kernel | Jupyter 셀을 실행하고 변수 상태를 유지하는 프로세스. |
| tensor / VRAM | 다차원 숫자 배열 / GPU가 사용하는 메모리. |
| parameter / gradient | 학습으로 바뀌는 모델 값 / 그 값을 바꿀 때 loss가 변하는 정도. |
| epoch / iteration | 학습 데이터를 한 번 도는 단위 / 한 번의 학습 업데이트. sampling에 따라 정의 확인. |
| train / validation / test | 가중치 학습 / 설정 선택과 점검 / 마지막 일반화 평가에 사용하는 구분. |
| policy / inference | 관측에서 행동을 정하는 규칙 또는 모델 / 모델 예측 실행. |
| imitation learning / BC | 시연으로 정책을 배우기 / 관측에서 시연 행동을 예측하는 행동 복제. |
| ACT / action chunk | Transformer로 행동 묶음을 예측하는 모델 / 연속 시점의 행동 시퀀스. |
| seed / reproducibility | 난수 생성의 시작 값 / 조건을 기록해 결과 재현을 돕는 것. |
| FPS / control frequency | 초당 영상 프레임 수 / 초당 제어 업데이트 횟수. 서로 같다고 가정하지 않음. |
| frame / pose / joint | 좌표계 또는 영상 한 장(문맥 확인) / 위치와 자세 / 로봇 관절. |
| latency / throughput | 요청 처리에 걸린 시간 / 단위 시간당 처리량. |
| API / schema / serialization | 프로그램 호출 계약 / 데이터 구조 / 데이터를 전송·저장 형식으로 바꾸기. |
| sim-to-real | 시뮬레이션에서 배운 것을 실제 로봇으로 옮기는 문제. 이번 단계의 완료 범위 밖. |

## 읽고 답해 보기

1. Python 확장과 Python 패키지는 어디가 다른가?
2. 왜 학습과 검증을 에피소드 단위로 나누는가?
3. loss 감소와 집기 성공률은 왜 다른가?
4. 이미지 shape, 행동 단위, 제어 주기가 왜 중요할까?
5. 아이패드 웹에서 RTX 3060 학습이 바로 되지 않는 이유는 무엇인가?

# 설치 보고서 - 주요 구성

| 구성 | 설치·확인 버전 | 역할 |
|---|---|---|
| VS Code | 1.140.0 | 편집기 |
| Python / uv | 3.12.15 / 0.12.22 | 프로젝트 실행 / 의존성 관리 |
| Git for Windows | 2.55.0.windows.5 | 코드 버전 관리 |
| PyTorch / torchvision | 2.11.0+cu128 / 0.26.0+cu128 | GPU 학습 / 이미지 모델 도구 |
| LeRobot / gym-hil | 0.6.1 / 0.1.14 | 로봇 학습 / 집기 환경 |
| MuJoCo / Gymnasium | 3.8.1 / 1.3.0 | 물리 엔진 / 환경 API |
| NumPy / pandas | 2.2.6 / 2.3.3 | 배열 / 표 분석 |
| SciPy / scikit-learn | 1.18.1 / 1.9.1 | 수치 계산 / 기초 ML 도구 |
| Matplotlib / ImageIO | 3.11.2 / 2.38.0 | 그래프 / 이미지·영상 읽기 |
| FastAPI / Uvicorn | 0.142.2 / 0.54.0 | 추론 API / 서버 |
| JupyterLab / ipykernel | 4.6.4 / 7.4.0 | 노트북 / Python 실행 커널 |
| Ruff / pytest | 0.16.10 / 9.1.1 | 코드 검사·서식 / 테스트 |
| FFmpeg shared / TorchCodec | 8.1.2 / 0.11.1 | 영상 프로그램 / 영상 로더 |

## 무엇을 검증했고 무엇은 남았나

18개 검사 항목 모두 통과: CUDA 학습 1회, 14개 주요 모듈 import, MuJoCo 물리·RGB, 집기 환경 reset/step, 영상 디코딩. uv의 의존성 호환 검사도 통과했습니다. 모델 학습 완료, 집기 성공, 추론 서버 배포, ROS 2 동작을 주장할 근거는 아직 없습니다.

전체 Python 패키지는 다음 2쪽에 기록했습니다. 직접 설치한 패키지와 자동으로 따라온 의존성을 모두 포함한 178개입니다. VS Code 확장, Git, FFmpeg, uv 실행 도구는 이 .venv 목록과 별개입니다.

정확한 버전의 기준은 uv.lock과 docs/installed-packages-windows.txt입니다. 향후 uv add/update를 실행하면 이 문서의 스냅샷과 달라질 수 있습니다. ROS 2, AWS 도구, LangGraph와 RAG 패키지는 이번 로봇 환경에 설치하지 않았습니다.

# 전체 Python 패키지 목록 1 / 2

2026-10-04 · 프로젝트 .venv · 설치된 배포 패키지 이름과 버전

| 패키지 = 버전 | 패키지 = 버전 |
|---|---|
| absl-py = 2.5.0 | GitPython = 3.2.0 |
| accelerate = 1.15.0 | glfw = 2.10.2 |
| aiohappyeyeballs = 2.7.1 | gym-hil = 0.1.14 |
| aiohttp = 3.14.3 | gymnasium = 1.3.0 |
| aiosignal = 1.4.0 | h11 = 0.16.0 |
| annotated-doc = 0.0.5 | hf-xet = 1.6.0 |
| annotated-types = 0.8.0 | hidapi = 0.15.0 |
| anyio = 4.15.1 | httpcore = 1.0.9 |
| argon2-cffi = 25.1.0 | httpx = 0.28.1 |
| argon2-cffi-bindings = 26.1.0 | huggingface_hub = 1.33.0 |
| arrow = 1.4.0 | idna = 3.20 |
| asttokens = 3.0.2 | ImageIO = 2.38.0 |
| async-lru = 2.3.0 | imageio-ffmpeg = 0.6.0 |
| attrs = 26.1.0 | iniconfig = 2.3.0 |
| av = 15.1.0 | ipykernel = 7.4.0 |
| babel = 2.18.0 | ipython = 9.17.1 |
| beautifulsoup4 = 4.15.0 | ipython_pygments_lexers = 1.1.1 |
| bleach = 6.4.0 | isoduration = 20.11.0 |
| certifi = 2026.7.22 | jedi = 0.20.0 |
| cffi = 2.1.1 | Jinja2 = 3.1.6 |
| charset-normalizer = 3.5.2 | joblib = 1.6.0 |
| click = 8.5.0 | json5 = 0.15.0 |
| cloudpickle = 3.1.2 | jsonlines = 4.0.0 |
| cmake = 4.1.3 | jsonpointer = 3.1.1 |
| colorama = 0.4.6 | jsonschema = 4.26.0 |
| comm = 0.2.3 | jsonschema-specifications = 2025.9.1 |
| contourpy = 1.4.0 | jupyter-events = 0.12.1 |
| cycler = 0.12.1 | jupyter-lsp = 2.3.1 |
| datasets = 4.8.5 | jupyter_builder = 1.2.3 |
| debugpy = 1.8.22 | jupyter_client = 8.10.0 |
| defusedxml = 0.7.1 | jupyter_core = 5.9.1 |
| dill = 0.4.1 | jupyter_server = 2.21.1 |
| draccus = 0.11.6 | jupyter_server_terminals = 0.5.4 |
| einops = 0.8.2 | jupyterlab = 4.6.4 |
| etils = 1.14.0 | jupyterlab_pygments = 0.3.0 |
| executing = 2.2.1 | jupyterlab_server = 2.28.1 |
| Farama-Notifications = 0.0.6 | kiwisolver = 1.5.1 |
| fastapi = 0.142.2 | lark = 1.3.1 |
| fastjsonschema = 2.22.2 | lerobot = 0.6.1 |
| filelock = 4.0.9 | MarkupSafe = 3.0.4 |
| fonttools = 4.66.1 | matplotlib = 3.11.2 |
| fqdn = 1.6.0 | matplotlib-inline = 0.2.2 |
| frozenlist = 1.8.0 | mergedeep = 1.3.4 |
| fsspec = 2026.2.0 | mistune = 3.3.4 |
| gitdb = 4.0.12 | mpmath = 1.3.0 |

# 전체 Python 패키지 목록 2 / 2

2026-10-04 · 프로젝트 .venv · 설치된 배포 패키지 이름과 버전

| 패키지 = 버전 | 패키지 = 버전 |
|---|---|
| mujoco = 3.8.1 | rfc3339-validator = 0.1.4 |
| multidict = 6.9.1 | rfc3986-validator = 0.1.1 |
| multiprocess = 0.70.19 | rfc3987-syntax = 1.1.0 |
| mypy_extensions = 1.1.0 | rpds-py = 2026.6.3 |
| narwhals = 2.26.0 | ruff = 0.16.10 |
| nbclient = 0.11.0 | safetensors = 0.8.0 |
| nbconvert = 7.17.1 | scikit-learn = 1.9.1 |
| nbformat = 5.11.1 | scipy = 1.18.1 |
| nest-asyncio2 = 1.7.3 | Send2Trash = 2.1.0 |
| networkx = 3.7 | sentry-sdk = 2.71.0 |
| notebook_shim = 0.2.4 | setuptools = 81.0.0 |
| numpy = 2.2.6 | six = 1.17.0 |
| opencv-python-headless = 4.13.0.92 | smmap = 5.0.3 |
| opentelemetry-api = 1.45.0 | soupsieve = 2.10 |
| packaging = 25.0 | stack-data = 0.6.3 |
| pandas = 2.3.3 | starlette = 1.7.0 |
| pandocfilters = 1.5.1 | sympy = 1.14.0 |
| parso = 0.8.7 | termcolor = 3.3.0 |
| pillow = 12.3.0 | terminado = 0.18.1 |
| platformdirs = 4.12.2 | threadpoolctl = 3.7.0 |
| pluggy = 1.6.0 | tinycss2 = 1.5.1 |
| prometheus_client = 0.26.0 | toml = 0.10.2 |
| prompt_toolkit = 3.0.53 | torch = 2.11.0+cu128 |
| propcache = 0.5.4 | torchcodec = 0.11.1 |
| protobuf = 7.36.2 | torchvision = 0.26.0+cu128 |
| psutil = 7.2.2 | tornado = 6.5.10 |
| pure_eval = 0.2.4 | tqdm = 4.70.1 |
| pyarrow = 25.0.1 | traitlets = 5.16.1 |
| pycparser = 3.0 | typing-inspect = 0.9.0 |
| pydantic = 2.13.5 | typing-inspection = 0.4.4 |
| pydantic_core = 2.46.5 | typing_extensions = 4.16.0 |
| pygame = 2.6.1 | tzdata = 2026.5 |
| Pygments = 2.21.0 | uri-template = 1.3.0 |
| pynput = 1.8.2 | urllib3 = 2.8.0 |
| PyOpenGL = 3.1.10 | uvicorn = 0.54.0 |
| pyparsing = 3.3.3 | wandb = 0.27.2 |
| pytest = 9.1.1 | wcwidth = 0.9.1 |
| python-dateutil = 2.9.0.post0 | webcolors = 25.10.0 |
| python-json-logger = 4.2.0 | webencodings = 0.6.1 |
| pytz = 2026.4 | websocket-client = 1.9.2 |
| pywinpty = 3.0.5 | xxhash = 4.0.1 |
| PyYAML = 6.0.3 | yarl = 1.25.1 |
| pyzmq = 27.2.0 | zipp = 4.1.0 |
| referencing = 0.37.0 | |
| requests = 2.34.2 | |

# 공식 자료와 다음 공부

설치 사실은 이 PC의 실행 결과와 패키지 메타데이터를 기준으로 작성했습니다. 아래 문서는 개념·사용법을 확인할 때 참고하는 공식 자료입니다. 링크를 눌러 열 수 있습니다. 문서 main/stable은 바뀔 수 있으므로 설치 버전도 함께 확인하세요.

VS Code - Python 환경 선택

https://code.visualstudio.com/docs/python/environments

VS Code - 표시 언어

https://code.visualstudio.com/docs/configure/locales

VS Code - 웹 편집기의 범위

https://code.visualstudio.com/docs/remote/vscode-web

JetBrains 계열 키맵 확장

https://marketplace.visualstudio.com/items?itemName=k--kato.intellij-idea-keybindings

uv - 프로젝트 파일 구조

https://docs.astral.sh/uv/concepts/projects/layout/

PyTorch - 기초 학습 흐름

https://docs.pytorch.org/tutorials/beginner/basics/intro.html

LeRobot - 설치

https://huggingface.co/docs/lerobot/v0.6.1/installation

ACT - 원 논문과 프로젝트

https://tonyzhaozh.github.io/aloha/

Gymnasium - 환경 API

https://gymnasium.farama.org/api/env/

MuJoCo - 공식 문서

https://mujoco.readthedocs.io/en/stable/overview.html

TorchCodec - 호환성과 설치

https://github.com/meta-pytorch/torchcodec

FastAPI - 처음 시작하기

https://fastapi.tiangolo.com/tutorial/first-steps/

ROS 2 - 기본 개념

https://docs.ros.org/en/rolling/Concepts/Basic.html

### 다음 공부의 출발점

첫 번째 목표는 11쪽 실습의 각 줄을 설명하는 것입니다. 그다음 집기 환경의 action_space와 observation_space를 확인하고, 시연 에피소드 하나를 저장하세요. ACT 전체 구조 공부는 실제 관측·행동 데이터를 본 뒤 시작하면 이해하기 쉽습니다.